

[🔗 If / Elif / Else](#)

INSTANT

[🔄 For & While Loops](#)[🌟 Data Analysis Diploma • Session 10](#)

PYTHON

CONTROL FLOW

Conditional Statements (if/elif/else) · Loops (for/while) · Loop Control

3 Modules

Core Topics

Conditionals • Loops • Control

14 Slides

Curriculum Depth

Python 3.12 Control Flow

Live Demos

Interactive Labs

Code Execution Engine

[Start Session 10 →](#)[⏏ Break & Continue](#)[>_ Logic & Functions](#)

— CURRICULUM OVERVIEW

What We'll Cover Today

Master decision making and iteration mechanisms in Python:

01

Conditional Statements

Learn how to make your code think and decide using if, elif, and else statements.

MODULE 1



02

Loops

Master repeating code efficiently with for loops and while loops.

MODULE 2



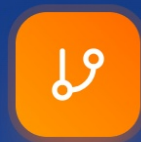
03

Loop Control

Fine-tune your loops with break, continue, and pass statements.

MODULE 3





01 CHAPTER ONE

Conditional Statements

Making Your Code Think & Decide

— WHY CONDITIONALS?

Why Do We Need **Conditionals**?

Making code intelligent, dynamic, and responsive to different situations:

✗ Without Conditionals

Code runs in a straight line — top to bottom. It cannot respond to different situations.

Every user sees the same result, no matter what.

✓ With Conditionals

Code becomes **intelligent** — it adapts to data and user input.

Different situations produce different results. Your program can make decisions!

THE CORE IDEA & INTERACTIVE LIGHT SIMULATOR

"If something is true, do this; otherwise, do that"

Green

Yellow

Red

if green → GO

— IF STATEMENT

The if Statement — Your First Decision

Syntax rules, condition evaluation, and mandatory 4-space indentation:

SYNTAX

```
if condition:
    # indented code block
```

How It Works

1. Python evaluates the **condition**
2. If **True** → executes the code block
3. If **False** → skips the code block



The colon (:) and indentation (4 spaces) are mandatory!

Indentation is Python's way of grouping code. It's not optional — it's the rule!

EXAMPLE: VOTING ELIGIBILITY

[Interactive Test](#)

age = 18

age >= 18: True

```
age = 18
if age >= 18:
    print("You can vote!")
```

TERMINAL OUTPUT:

✓ "You can vote!"

What's Happening? We set **age = 18**, then check **if age >= 18**. Since 18 is equal/greater than 18, condition is True, so Python prints message.

— ELSE & ELIF

else & elif — Handling Multiple Paths

Chaining conditions and providing catch-all fallback execution:

if (condition 1) → elif (condition 2) → else

Only one block executes – top to bottom

else

Catches everything the **if** didn't handle. Think: "otherwise, do this."

```
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

elif (else if) & Live Grade Simulator

score = 85

grade = "B"

```
if score >= 90: grade = "A"
elif score >= 80: grade = "B"
elif score >= 70: grade = "C"
else: grade = "F"
```

Why It Matters & Rule of Thumb: Real-world scenarios have multiple outcomes. Start with **if**, add **elif** for extra conditions, and finish with **else** as the catch-all.

— NESTED IF & ERRORS

Nested Conditionals & Common Mistakes

Multi-level logic hierarchy, syntax traps, and debugging strategies:

Nested if Statements

An **if** inside another **if** — for multi-level decisions.

☒ logged_in = True

role = "admin" ▼

```
if logged_in:
    if role == "admin":
        print("Admin panel")
    else:
        print("User panel")
```

RESULT:

► Output: "Admin panel"

COMMON ERRORS

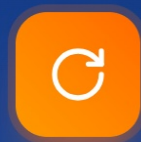
⚠ Forgetting the colon (:) after if/elif/else

⚠ Wrong indentation — mixing spaces and tabs

⚠ Using = (assignment) instead of == (comparison)

BEST PRACTICES

1. Keep nesting shallow — max 2–3 levels deep.
2. Use **elif** instead of deep nesting when possible.
3. Read error messages carefully to locate syntax bugs.
4. Test with edge cases to ensure logical correctness.



02 CHAPTER TWO

Loops

Repeating Code Efficiently

for Loop — Iterate with Precision

Sequence iteration, range() parameters, and string processing:

Syntax & How It Works

```
for variable in sequence:
    # code to repeat
```

1. The sequence provides items one by one
2. Each item is assigned to variable
3. Perfect when you know what you're iterating over

The range() Function

```
range(5) # 0, 1, 2, 3, 4
range(1, 6) # 1, 2, 3, 4, 5
range(0, 10, 2) # 0, 2, 4, 6, 8
```

Interactive for Loop Animator

▶ **Step (1/5)**

```
for i in range(1, 6):  
    print(i)
```

5

word = "Python" → **Current letter: 'P'**

— WHILE LOOP

while Loop — Repeat Until Ready

Condition-driven loops, countdown timers, input validation, and game loops:

Syntax & How It Works

```
while condition:
    # code to repeat
```

1. Python checks condition
2. If True → runs code block & checks again
3. If False → exits the loop



Danger: Infinite Loop!

If condition never becomes False, code repeats forever. Always ensure condition can change!

for vs while: Use **for** when count is known. Use **while** when count is unknown.

Countdown Timer Demo

Step (count = 5)

```
count = 5
while count > 0:
    print(count); count -= 1
```

User Input Validation Demo

Type "secret" to unlock...

```
while password != "secret": prompt input...
```

— BREAK, CONTINUE & PASS

break, continue, pass — Fine Control

Exiting early, skipping iterations, and placeholder statements in loops:

break

continue

pass

break

Immediately exits the entire loop. Use when you've found what you need.

```
for n in [1,3,4,5]:
    if n % 2 == 0:
        print(n) # 4
        break
```

"I'm done, stop everything!"

continue

Skips current iteration and moves to the next one immediately.

```
for i in range(6):
    if i % 2 != 0:
        continue
    print(i) # 0 2 4
```

"Skip this one, keep going!"

pass

Does absolutely nothing. A placeholder for future code stubs.

```
if user == "admin":
    pass # TODO later
else:
    print("No access")
```

"I'll come back to this later."

— NESTED LOOPS & PITFALLS

Nested Loops & Common Errors

Outer and inner loop iterations, matrix traversal, and complexity:

What Are Nested Loops?

A loop inside another loop. The inner loop completes all iterations for each single iteration of the outer loop.

```
for i in range(3): # outer
    for j in range(3): # inner
        print(f"{i},{j}")
# Runs 3 x 3 = 9 times!
```

Current: i=0, j=0

Step Next Iteration

(0,0)

(0,1)

(0,2)

(1,0)

(1,1)

(1,2)

(2,0)

(2,1)

(2,2)

Use Cases

- Pattern printing — stars, triangles
- Matrix traversal — rows and columns
- Combinations — finding pairs

Performance note: Nested loops multiply iterations: $3 \times 3 = 9$, $10 \times 10 = 100$!

Common Errors & Tips

Mistake 1: Forgetting to update loop variable → infinite while loop

Mistake 2: Off-by-one in range() — range(5) gives 0–4, not 1–5

Tip: Use meaningful names i, j and track 4-space indentation carefully!

Your Control Flow Toolkit

Complete reference guide to Python decision logic and loop control:

Conditionals

if / elif / else

Make decisions in your code. Handle multiple outcomes with clean logic.

for Loop

```
for x in sequence:
```

Iterate over known collections. Perfect when count is known.

while Loop

while condition:

Repeat until condition changes. Great for unknown iteration counts.

break

Exit the loop early when you've found what you need.

continue

Skip current iteration and move directly to the next one.

pass

Placeholder that does nothing. Use for future code stubs.

Practice makes perfect — start with small examples, then build bigger!

INSTANT

✔ SESSION 10 COMPLETED SUCCESSFULLY

THANK YOU

Thank you for your active participation!

Next Session: Python Data Structures – Lists, Tuples, Dictionaries & Sets

↺ Restart Presentation

Open Session 11: Python Data Structures →